

Semi-Digitaler Controller für eine analoge Anlage



Projekt: Semi-digitaler Controller für eine analoge Modellbahnanlage
Projektart: Sonderleistung / Projektarbeit zur Notenaufbesserung
Vorgelegt; Jonas Israel
Datum: 10.06.2026

Warum & Konzeption (Projektbegründung)	3
Warum die Eigenentwicklung?	3
Eigenbau-Analog vs. Kommerzielle Digitalsteuerung (DCC)	3
Bereits existierende Systeme mit DCC-Standard	3
Die Eigenbau-Analogsteuerung (ESP32-Lösung)	4
Fazit des Systemsvergleichs	5
Hardware & Schaltungsdesign	5
MOSFET vs. H-Brücke	5
Der einzelne MOSFET	6
Die H-Brücke	6
Fazit und Bauteilauswahl	6
Technische Zeichnung der finalen Schaltung mit dem ESP32.	7
Vorwärts	8
Rückwärts	9
Komponenten Liste	9
Test- und Demoumgebung	10
Umsetzung	10
Aufziehen eines Einfachen Webserver um zu Testen	10
Verkabelung	11
PWM-Integration	11
Realisation und Änderungen	11
JSON Datenbank	12
Dynamische Tabelle und Verwaltungs-Overlay	13
Finale Integration der H-Brücke	13
Das Frontend	14
Einschränkungen	14
IT Security	14
Fazit & Ausblick	15

Warum & Konzeption (Projektbegründung)

Warum die Eigenentwicklung?

Herkömmliche analoge Fahrpulte für die Modelleisenbahn erfordern physischen Zugriff. Da ich meine Lokomotiven jedoch gerne automatisiert laufen lasse, während ich an der Anlage arbeite und im Zugriffsloch stehe, sind die Transformatoren für mich schwer erreichbar. Da gerade bei neuen Anlagen häufig Fehler wie Entgleisungen oder Kurzschlüsse auftreten, ist ein schneller Zugriff auf die Steuerung zwingend erforderlich, um Schäden zu vermeiden.

Hierbei hatte ich die Wahl, auf bestehende Systeme zurückzugreifen oder eine Eigenbau-Lösung zu realisieren. Ich habe mich für den Eigenbau entschieden, da mein bisheriges Rollmaterial vollständig analog ist und mir die Zeit, das Budget sowie die Motivation fehlen, die teilweise historischen Lokomotiven auf den modernen DCC-Digitalstandard umzurüsten.

Eigenbau-Analog vs. Kommerzielle Digitalsteuerung (DCC)

Vor der praktischen Umsetzung stand die Überlegung im Raum, ein teures Standard-Digitalsystem auf DCC-Basis anzuschaffen. Um die Entscheidung gegen ein solches System und für die vorliegende Eigenentwicklung zu begründen, werden im Folgenden die Vor- und Nachteile beider Ansätze im Stil einer Erörterung gegenübergestellt.

Bereits existierende Systeme mit DCC-Standard

- **Vorteile:**
 - **Plug and Play:** Die großen Standardlösungen der etablierten Hersteller können unkompliziert installiert werden und sind sofort funktionsbereit.
 - **Geringe Einstiegshürde:** Für die Inbetriebnahme ist nur ein moderates technisches Grundwissen erforderlich. Es muss keine eigene Software geschrieben oder komplexe Schaltungen entworfen werden.
 - **Zukunftssicherheit:** Neue Generationen von Triebfahrzeugen setzen fast ausschließlich auf digitale Standards wie DCC. Reine Analogmodelle werden von den Herstellern kaum noch produziert.
- **Nachteile:**
 - **Hohe Anschaffungskosten:** Die Anschaffung von Digitalzentralen, Boostern und Handreglern ist mit erheblichen Investitionen verbunden.
 - **Aufwendiger Umbau von Fahrzeugen und Stromkreisen:** Da die bestehende Anlage historisch analog aufgebaut ist, müsste das Schienennetz für den Digitalbetrieb umfassend modernisiert werden (z. B. durch komplexe Kehrschleifenmodule und angepasste Stromeinspeisungen). Zudem muss der vorhandene Fuhrpark auf DCC umgerüstet werden. Dies erfordert den

Kauf von Decodern und einen oft schwierigen Einbau, der bei der Spur N (1:160) von einfachem Stecken bis hin zum mühsamen Fräsen des Lokgehäuses, Löten und Isolieren auf engstem Raum reicht.

- **Herstellerbindung und mangelnde Freiheit:** Mit der Entscheidung für eine bestimmte Zentrale (wie z. B. die z21 von Roco) bindet man sich an das geschlossene Ökosystem des jeweiligen Herstellers. Dies betrifft nicht den offenen DCC-Übertragungsstandard an sich, sondern die Hardware: Regler, Weichendecoder und Lichtsteuerungen müssen mit dem gewählten System kompatibel sein, wodurch man auch an alle softwareseitigen Limitationen des Herstellers gebunden ist.
- **Steigende Lokpreise:** Da moderne Lokomotiven herstellerseitig oft direkt mit Sound- und Digitaldecodern ausgestattet sind, steigt die Komplexität der Fahrzeuge – und damit auch deren Neupreis im Handel massiv.

Die Eigenbau-Analogsteuerung (ESP32-Lösung)

- **Nachteile:**

- **Enormer Entwicklungsaufwand:** Da es sich um eine komplette Eigenentwicklung handelt, müssen sämtliche Aspekte – von der Recherche über das Schaltungsdesign bis hin zur Programmierung – selbst übernommen werden. Der Aufwand bis zur Inbetriebnahme ist dadurch massiv höher.
- **Erhöhtes technisches Verständnis erforderlich:** Die Installation verzeiht keine Fehler. Das eigenständige Verdrahten von Leistungselektronik und das Schreiben von Firmware setzen ein tiefes Verständnis von Mikrooptokopplern, Signalverarbeitung und Programmierung voraus, was deutlich komplexer ist als eine fertige Kauflösung.
- **Gleisgebundene Einschränkung:** Das System nutzt keine selektive Adressierung der einzelnen Fahrzeuge, sondern das klassische analoge Prinzip („Spannung auf dem Gleis lässt die Lok fahren“), gesteuert über ein digitales Interface. Zwei Lokomotiven auf demselben Gleisabschnitt lassen sich daher nicht unabhängig voneinander steuern, sondern reagieren zeitgleich, sofern das Layout nicht über isolierte Gleisabschnitte unterteilt wird.

- **Vorteile:**

- **Geringe Materialkosten:** Die Anschaffungskosten sind extrem niedrig, da nur die tatsächlich benötigten Bauteile beschafft werden müssen. Da ein analoger Fahrstrom-Trafo meist schon vorhanden ist, beschränken sich die Zusatzkosten im Wesentlichen auf günstige Komponenten wie den ESP32-Mikrocontroller und die H-Brücke.
- **Kein Umbau des Fuhrparks nötig:** Die Lokomotiven können in ihrem Originalzustand verbleiben. Das spart nicht nur erhebliche Kosten für Decoder, sondern senkt auch den Zeitaufwand bei der Fahrzeugwartung gegen null.
- **Erhalt von Sammlerwert und Historie:** Ältere historische Lokomotiven (beispielsweise Modelle aus DDR-Produktion) können absolut originalgetreu im Werkszustand belassen werden. Dadurch bleibt ihr Sammler- und historischer Wert vollständig erhalten, während sie durch die moderne

PWM-Ansteuerung des ESP32 dennoch von den Fahreigenschaften moderner Digitaltechnik (wie feinfühligem Anfahren) profitieren.

- **Wirtschaftlicherer Fuhrpark-Ausbau:** Auf dem Gebrauchtmrkt existiert ein riesiges und oft sehr günstiges Angebot an analogen Spur-N-Lokomotiven, die von Modellbahnern weit unter ihrem ursprünglichen Wert abgegeben werden. Diese können ohne Folgekosten direkt auf der Anlage eingesetzt werden.
- **Maßgeschneiderte Anpassung an eigene Bedürfnisse:** Das System bietet absolute Freiheit und lässt sich perfekt auf die Gegebenheiten der eigenen Anlage zuschneiden. Zukünftige Erweiterungen und Automatisierungen – wie etwa eine Raumlicht-abhängige Anlagenbeleuchtung oder automatisierte Schattenbahnhöfe – lassen sich flexibel über die Software implementieren, ohne teure Zusatzmodule kaufen zu müssen.

Fazit des Systemsvergleichs

Wie die Gegenüberstellung der Vor- und Nachteile deutlich zeigt, bietet meine Eigenbau-Lösung einen enormen finanziellen Vorteil. Zudem ermöglicht sie es mir, meinen bestehenden Fuhrpark und insbesondere die historischen Lokomotiven im Originalzustand zu belassen – ein Punkt, der für mich persönlich und für den Werterhalt der Modelle von zentraler Bedeutung ist.

Gleichzeitig macht der Vergleich deutlich, dass kommerzielle DCC-Standards durchaus ihre Berechtigung haben, da sie einen unkomplizierten Einstieg bieten und für viele Modellbahner genau die richtige Lösung sind. Für dieses Projekt stand jedoch nicht die einfachste Kauflösung im Vordergrund: Ich realisiere diese Eigenentwicklung aus Freude am Basteln und mit dem klaren Ziel, mein technisches Wissen praxisnah zu erweitern und dadurch ein System mit echtem, individuellem Mehrwert zu schaffen.

Hardware & Schaltungsdesign

MOSFET vs. H-Brücke

Vorab ist anzumerken, dass Motorsteuerungen in der Praxis meist entweder aus Relais oder diskreten MOSFETs aufgebaut werden. Da eine elektronische H-Brücke im Grunde aus einer intelligenten Verschaltung mehrerer MOSFETs besteht, vergleicht dieses Kapitel den Einsatz eines einzelnen MOSFETs mit dem einer vollständigen H-Brücke, um die finale Bauteil Auswahl für dieses Projekt zu begründen.

Der einzelne MOSFET

Ein MOSFET (Metall-Oxid-Halbleiter-Feldeffekttransistor) fungiert als elektronischer Schalter. Mit ihm lassen sich Lasten, die eine hohe Stromstärke und Spannung benötigen

(wie der Fahrstrom der Modellbahn), über eine niedrige Logikspannung (3,3V des ESP32) steuern. Für den Lastkreis ist dennoch eine externe Spannungsquelle (in diesem Fall der 12V-Trafo) erforderlich.

Vorteile:

- Sehr geringe Bauteilkosten.
- Einfacher, platzsparender Schaltungsaufbau.

Nachteile:

- Keine Richtungsumkehr: Ein einzelner MOSFET kann den Strom nur ein- und ausschalten. Die Polarität am Gleis bleibt immer gleich, wodurch sich der Motor nur in eine Richtung drehen kann.

Die H-Brücke

Die H-Brücke nutzt vier kreuzweise angeordnete MOSFETs als Schalter. Der Name leitet sich von der Anordnung der Komponenten ab, die im Schaltplan der Form des Buchstabens „H“ gleicht. Da sie auf denselben Halbleiterkomponenten basiert, teilt sie die grundlegenden Eigenschaften des einzelnen MOSFETs (schnelles Schalten, geringe Verluste), erweitert diese jedoch um eine entscheidende Funktion.

Vorteil:

- Volle Richtungskontrolle: Durch das gezielte, diagonale Schalten der vier internen MOSFETs kann die Polarität am Gleis im laufenden Betrieb getauscht werden. Dies ermöglicht den Vorwärts- und Rückwärtslauf des Zuges.

Nachteil:

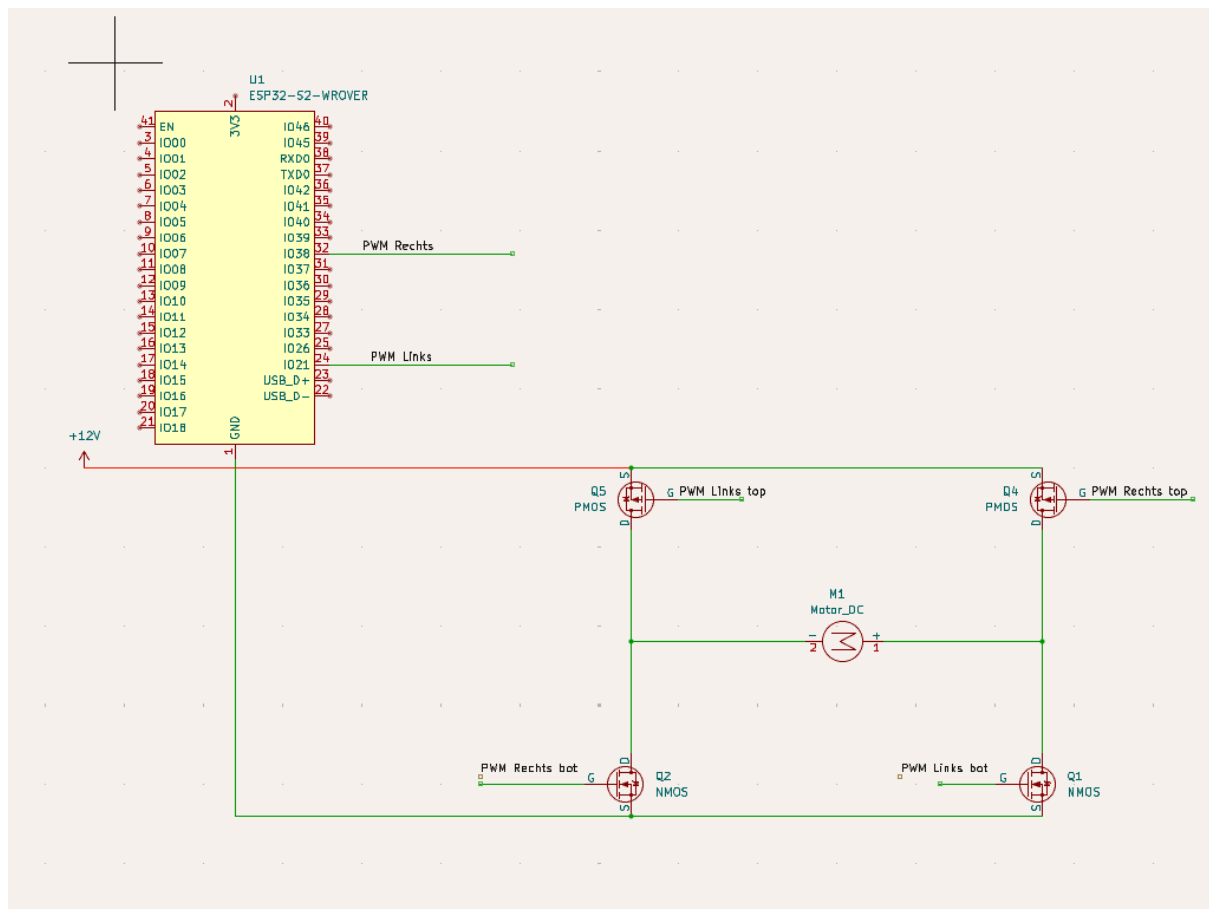
- Höhere Anschaffungskosten bei fertigen Modulen.
- Sehr komplexer Eigenbau, da Schutzbeschaltungen (z. B. gegen Kurzschlüsse beim Umschalten) integriert werden müssen.

Fazit und Bauteilauswahl

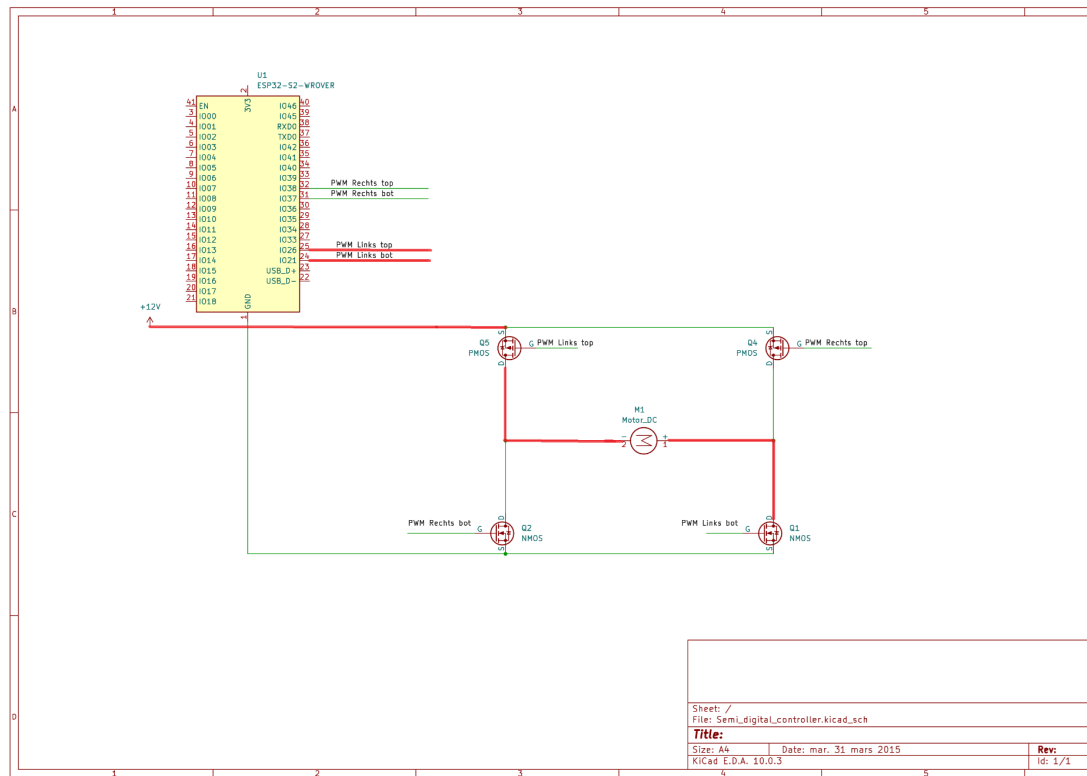
Aufgrund der zwingenden Notwendigkeit, die Züge auf der Spur-N-Anlage in beide Fahrtrichtungen steuern zu können, fiel die Entscheidung gegen einen einzelnen MOSFET und für eine H-Brücke.

Um ein robustes Gesamtsystem zu garantieren und den Eigenaufwand bei den Schutzschaltungen zu minimieren, wurde das fertige Treibermodul **IBT_2** (basierend auf dem BTS7960-Chipsatz) gewählt. Dieses ist für die Ströme der Spur-N-Anlage zwar überdimensioniert, bietet dadurch aber eine extrem hohe Ausfallsicherheit und thermische Stabilität im Dauerbetrieb.

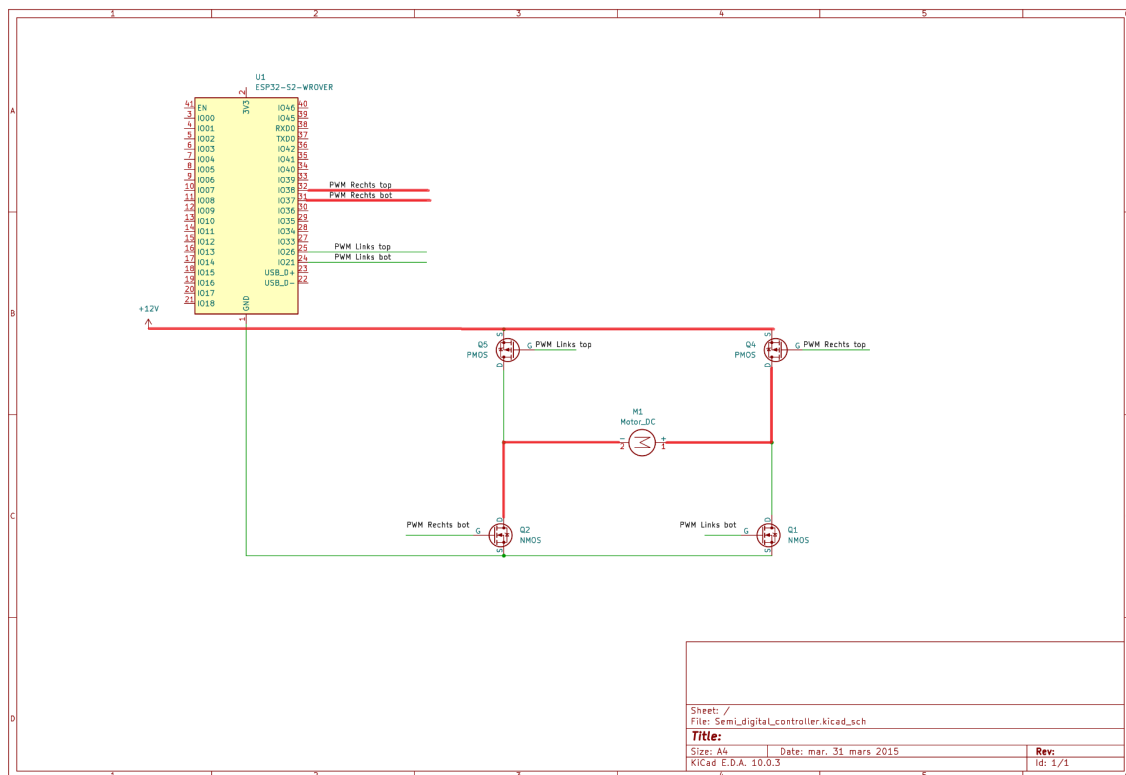
Technische Zeichnung der finalen Schaltung mit dem ESP32.



Vorwärts



Rückwärts



Komponenten Liste

Um das Projekt erfolgreich nachzubauen oder zu erweitern, werden die nachfolgend aufgeführten Komponenten benötigt.

Wichtiger Hinweis zum System: Die gesamte Schaltung und Steuerung basiert rein auf Gleichstrom (DC). Das System ist für das klassische Zwei-Leiter-System ausgelegt und nicht mit dem Drei-Leiter-Wechselstrom-System (AC) der Firma Märklin kompatibel.

Komponente	Funktion im Projekt	Empfehlung / Spezifikation
Mikrocontroller	Hauptsteuerung (Webserver, Logik, PWM)	ESP32-WROVER (aufgrund des zusätzlichen PSRAMs für den Webserver)

H-Brücken-Treiber	Leistungsstufe zur Motor- und Richtungskontrolle	IBT_2 (BTS7960-Chipsatz)
Speichermodul	Dynamic Configuration: Auslesen von fahrzeugspezifischen Konfigurationsdaten (JSON-Format) beim Systemstart.	SD-Karten-Reader (SPI-Schnittstelle)
Externe Stromquelle	Versorgung der Gleise und der Leistungselektronik	12V - 14V DC Netzteil (Gleichspannung)
Labora Aufbau	Temporäre Verkabelung und Prototyping	Breadboard und Jumperkabel (Male-Male / Male-Female)

Test- und Demoumgebung

- Gleismaterial: Standard-Gleichstromgleise (z. B. Spur N) zum Aufbau eines Testkreises.
- Triebfahrzeug: Eine analoge Gleichstrom-Lokomotive (DC) zu Testzwecken.

Umsetzung

Aufziehen eines Einfachen Webserver um zu Testen

Nachdem ich die passenden Bauteile herausgesucht hatte, war der erste praktische Schritt im Projekt der Aufbau einer sehr einfachen Webseite mit den nötigsten Steuerungselementen. Diese Webseite hat als Werkzeug gedient, um die spätere Steuerungslogik und die Hardware-Verbindungen direkt im echten Betrieb testen zu können.

Für die Oberfläche habe ich nur HTML und JavaScript benutzt, da ich hier schon gute Vorkenntnisse aus der Schule hatte und der Fokus erst einmal rein auf der Funktion lag.

Die Webseite wird direkt auf dem ESP32 gehostet. Um die Netzwerkverbindung und den Server einzurichten, kamen zwei wichtige Bibliotheken zum Einsatz:

- **WiFi.h:** Diese Bibliothek wurde genutzt, um den ESP32 mit dem WLAN zu verbinden. Ich habe mich für diese Standard-Bibliothek entschieden, da wir schon im Schulunterricht damit gearbeitet haben und ich das Wissen direkt nutzen konnte.
- **ESPAsyncWebServer.h:** Für den Webserver habe ich absichtlich eine asynchrone Variante gewählt. Der Hauptgrund dafür war die gute Dokumentation und die vielen hilfreichen Beiträge in Entwickler-Foren. Für den Einstieg in dieses Thema war das ein riesiger Vorteil. Außerdem hat ein asynchroner Server den Vorteil, dass er den ESP32 nicht blockiert, wenn Daten gesendet werden. Dadurch läuft die Steuerung der Züge später flüssig und ohne Unterbrechungen weiter.

Verkabelung

Nachdem die Webseite im lokalen Netzwerk erreichbar war, habe ich mit der Verkabelung des MOSFETs, der externen Stromquelle und der Gleise begonnen. Als Stromquelle nutze ich den analogen Geschwindigkeitsregler (den ich zur Einfachheit im Folgenden als Trafo bezeichne). Die Gleise sind in der technischen Zeichnung als Motor eingezeichnet, da am Ende die Lokomotive den eigentlichen Verbraucher darstellt.

PWM-Integration

Nach der Verkabelung habe ich das Vorwärtsfahren der Loks umgesetzt. Dies wurde über PWM (Pulsweitenmodulation) realisiert. Ich musste PWM verwenden, da ich mit dem Mikrocontroller keine variable analoge Spannung an die Gleise ausgeben kann, wenn die Eingangsspannung vom Trafo fest bei 12 bis 14V liegt. Deswegen schaltet der ESP32 den Strom in einem bestimmten Zeitabstand immer ganz schnell an und aus. Der Durchschnitt dieser Impulse ergibt dann die gewünschte Spannung am Gleis – so funktioniert kurz gesagt PWM.

Damit ich das Ganze steuern konnte, musste ich auch meine einfache Testseite anpassen. Das Eingabefeld für die Geschwindigkeit musste den Wert an den Mikrocontroller senden, ohne dass die komplette Seite neu geladen wird. Dafür verwende ich die `fetch()`-Funktion in JavaScript, die automatisch im Hintergrund ausgelöst wird, sobald der Wert im Feld geändert wird.

Bei den ersten Tests ist mir aufgefallen, dass jede Lokomotive bei den Standard-Einstellungen von PWM ein leichtes, hochfrequentes Piepsen von sich gegeben hat. Um dieses Geräusch zu eliminieren, habe ich die PWM-Frequenz im Code auf 16 kHz erhöht. Durch diese höhere Frequenz vibrieren die Motoren der Loks in einem Bereich, den das menschliche Ohr kaum noch oder gar nicht mehr wahrnimmt, wodurch das Piepsen nicht mehr zu hören war.

Realisation und Änderungen

Bei den gleichen ersten Tests, die ich im vorherigen Kapitel beschrieben habe, ist mir ein entscheidender Fehler aufgefallen. Da ich mich anfangs zu sehr auf den einzelnen MOSFET konzentriert hatte, habe ich übersehen, dass ich mit dieser ursprünglichen Konfiguration die Lokomotiven nur in eine einzige Richtung steuern konnte. Ein Rückwärtsfahren war technisch nicht möglich.

Nach längerer Recherche im Internet und mithilfe von KI kam ich zu dem Ergebnis, dass ich stattdessen eine H-Brücke benötige. Da ich mir im Vorfeld ein Set aus reinen N-Kanal-MOSFETs gekauft hatte, war es mir nicht möglich, eine eigene H-Brücke zu bauen. Aus diesem Grund habe ich mich für den Kauf eines fertigen Moduls entschieden. Wie bereits im Kapitel „MOSFET vs. H-Brücke“ theoretisch beschrieben, fiel die Wahl auf das Modul **IBT_2**.

Dieser Fehler hatte jedoch einen großen Vorteil: Durch den ersten Prototyp mit dem MOSFET konnte ich isoliert und in einer einfachen Umgebung ein grundlegendes Verständnis für PWM entwickeln. Dieses Wissen war für den anschließenden Umbau und die Integration der H-Brücke absolut notwendig und hat mir die weitere Arbeit massiv erleichtert.

JSON Datenbank

Während ich auf die Lieferung der neuen Komponenten (die H-Brücke) gewartet habe, wollte ich die Zeit sinnvoll nutzen. Der größte Fehler in einem Projekt wäre es schließlich, nichts zu tun. Also habe ich mir überlegt, welches Feature nicht nur cool, sondern auch richtig praktisch für die Modellbahn wäre. Spoiler vorab: Aufgrund einer für mich sehr steilen Lernkurve war dieser Teil anfangs ziemlich schmerzhaft zu entwickeln.

Die Idee war, eine lokale Datenbank auf Basis einer JSON-Datei aufzubauen. In dieser Datei werden wichtige Werte der Lokomotiven wie eine eindeutige ID, die Bezeichnung (Typ) und die Höchstgeschwindigkeit gespeichert.

Die Datenstruktur habe ich wie folgt aufgebaut:

```
{
  "Train": [
    {
      "id": 1,
      "type": "SD45",
      "max_speed": 120
    }
  ]
}
```

Diese JSON-Datei ist auf einer separaten Micro-SD-Karte gespeichert. Das hat den riesigen Vorteil, dass ich bei Änderungen an den Lokdaten nicht jedes Mal ein neues Dateisystem-Image (Fileimage) erstellen und auf den ESP32 flashen muss. Ich hatte hierbei großes Glück, da mein ESP32-WROVER-Modul bereits einen eingebauten SD-Kartenleser besitzt.

Für die Umsetzung musste ich zwei neue Bibliotheken in den Code einbinden:

- **SD_MMC.h**: Diese Bibliothek ist notwendig, um die Dateien auf der SD-Karte über den schnellen MMC-Modus des ESP32 zu verwalten. Das bedeutet, Daten von der Karte zu lesen, darauf zu schreiben, Dateien hinzuzufügen oder zu löschen.
- **ArduinoJson.h**: Da das reine Parsen (Auslesen) von Textdateien im Mikrocontroller sehr aufwendig ist, vereinfacht mir diese Bibliothek die Verarbeitung der gelesenen JSON-Datei massiv.

Die eingelesenen Werte der Loks werden auf der Hauptseite in einem Dropdown-Menü angezeigt. Wählt man eine Lok aus, schränkt das System automatisch die maximale Geschwindigkeit im Kontrollfeld ein, damit die Lok nicht schneller fährt, als in der Datenbank hinterlegt.

Zusätzlich habe ich eine separate Verwaltungsseite entwickelt. Diese generiert mithilfe von JavaScript automatisch eine dynamische Tabelle basierend auf den JSON-Werten. Diese Seite dient dazu, die Werte der Loks (CRUD-Operationen) direkt im Browser zu bearbeiten, neu anzulegen oder zu löschen – ohne dass man jedes Mal die SD-Karte physisch entnehmen und die Datei manuell am PC anpassen muss.

Dynamische Tabelle und Verwaltungs-Overlay

Um die Tabelle zu erstellen, nutze ich die `fetch()`-Funktion von JavaScript. Diese holt die Daten asynchron vom ESP32, sobald die Seite im Browser geladen wird. Der ESP32 muss die JSON-Datei hierfür lediglich von der SD-Karte einlesen und die Rohdaten direkt weitergeben.

Im JavaScript-Code laufe ich in einer Schleife (*Loop*) durch alle Einträge und erstelle für jede Lokomotive dynamisch ein Tabellenzeilen-Objekt (`<tr>`). Dieses wird anschließend per `appendChild()` an das übergeordnete Tabellen-Objekt angehängt (*append*), sodass alle neuen Werte sauber untereinander hinzugefügt werden.

Für die Verwaltung der Daten habe ich eine Bearbeitungsoberfläche gebaut. Diese ist als modales Overlay (ein Pop-up-Fenster über der Seite) realisiert, welches für jeden Wert (ID, Bezeichnung, Höchstgeschwindigkeit) ein Eingabefeld bereitstellt. In dieser Ansicht kann man bereits existierende Einträge bearbeiten, löschen oder komplett neue Lokomotiven hinzufügen.

Finale Integration der H-Brücke

Die H-Brücken-Module kamen genau zum richtigen Zeitpunkt an, als ich so gut wie fertig mit der JavaScript-Tabelle war. Ich habe die alte, provisorische MOSFET-Schaltung entfernt und mich gründlich über die Pin-Belegung des IBT_2-Moduls informiert, um eine korrekte Verkabelung zu garantieren.

Nach der neuen Verkabelung – die durch das fertige Modul im Vergleich zum Eigenbau deutlich einfacher war – habe ich den Code angepasst. Die Steuerung wurde auf zwei PWM-Pins umgebaut, um sowohl das Vorwärts- als auch das Rückwärtsfahren zu ermöglichen.

Nach kurzem Experimentieren mit einer Schalter-Checkbox zur Richtungsauswahl auf der Webseite habe ich mich aus pragmatischen Gründen für die einfachere und intuitivere Methode entschieden: Zwei separate Buttons für „Vorwärts“ und „Rückwärts“. Das verringert Fehlbedienungen und macht das Interface im Realbetrieb deutlich robuster.

Das Frontend

Da mein Fokus im Berufsalltag hauptsächlich auf der SAP-Entwicklung liegt und ich privat eher mit Unity und Konsolenanwendungen arbeite, hatte ich im Bereich der Web-Oberflächen weniger Erfahrung. Während des Projekts stand ich daher im Austausch mit Freunden und Kollegen, die mehr Erfahrung in der Frontend-Entwicklung haben. Sie haben mir mit wertvollen Tipps und Code-Schnipseln geholfen, die Oberfläche zu optimieren.

Das eigentliche Styling der Webseite wurde vollständig mit CSS umgesetzt, nachdem die Hardware-Ansteuerung und die Backend-Logik stabil liefen. Bei der Gestaltung habe ich eine „**Mobile First**“-Strategie verfolgt, damit sich die Modellbahn später bequem über das Smartphone steuern lässt.

Zudem lag der Fokus darauf, die Bedienung so einfach und intuitiv wie möglich zu machen:

- **Slider statt Eingabefelder:** Die Geschwindigkeit wird über Schieberegler (Slider) gesteuert, da das Eintippen von Zahlen im laufenden Fahrbetrieb unpraktisch ist.
- **Visuelles Feedback:** Die aktuell ausgewählte Fahrtrichtung (Vorwärts/Rückwärts) sowie der aktive Zustand werden im Interface optisch hervorgehoben, um Fehlbedienungen zu vermeiden.

Einschränkungen

Ich habe das Projekt bei der Architektur bewusst auf den ESP32 eingeschränkt. Das bedeutet, dass sowohl das Hosten der Webseite als auch die JSON-Datenbank direkt lokal auf dem Mikrocontroller laufen.

Obwohl ich im Vorfeld die Möglichkeit in Betracht gezogen habe, einen separaten SQL-Server und die Webseite auf einem externen Server aufzusetzen, habe ich mich dagegen entschieden. Da dieses Projekt als **Proof of Concept** (Machbarkeitsnachweis) dient, hätte eine solche Server-Infrastruktur den Rahmen gesprengt. Der zusätzliche Konfigurations- und Zeitaufwand wäre für diese Phase zu hoch gewesen. Durch die All-in-one-Lösung auf dem ESP32 konnte ich mich voll und ganz auf die stabile Kernfunktion – die Loksteuerung – konzentrieren.

IT Security

Da mein Schwerpunkt nicht in der IT-Sicherheit liegt, habe ich mich bei diesem Prototyp auf die grundlegenden und wichtigsten Absicherungen für den lokalen Betrieb konzentriert.

Die Webseite wird über das unverschlüsselte HTTP-Protokoll gehostet. Aus diesem Grund ist es absolut nicht vorgesehen, Ports im Router zu öffnen, um die Seite über das Internet erreichbar zu machen. Das stellt im geplanten Betrieb jedoch kein Sicherheitsrisiko dar, da das System als reine **On-Premises-Anwendung** (lokale Anwendung) konzipiert ist.

Die Sicherheit wird hierbei durch zwei Maßnahmen gewährleistet:

- **Netzwerk-Isolation:** Da keine Verbindung nach außen zum weltweiten Internet besteht, ist das System vor externen Angriffen geschützt.
- **Lokaler Zugriffsschutz:** Der Zugriff auf die Steuerung ist nur für Geräte möglich, die sich im selben lokalen Netzwerk befinden. Da das Netzwerk selbst (z. B. das heimische WLAN) verschlüsselt ist, ist die Anwendung für den privaten Gebrauch ausreichend abgesichert.

Fazit & Ausblick

Obwohl mein Projekt sicherlich nicht die universelle Lösung für jeden Modellbahner ist, bietet es einen entscheidenden Vorteil: Es ist die perfekte Lösung für Sammler, die den Originalzustand ihrer wertvollen analogen Lokomotiven erhalten wollen, aber trotzdem nicht auf den Komfort einer modernen, digitalen Steuerungsoberfläche verzichten möchten. Ein weiterer großer Pluspunkt ist die Flexibilität: Das System kann jederzeit komplett individuell an die Gegebenheiten meiner eigenen Anlage angepasst werden.

Das Projekt hatte – wie jede echte Entwicklung – seine Höhen und Tiefen. Für mich persönlich ist das Ergebnis extrem praxistauglich. Besonders wichtig ist mir hierbei der Gedanke, dass der aktuelle Projektstand niemals der endgültige Endstand sein wird.

Für die Zukunft sind bereits die nächsten Funktionen und Erweiterungen fest in Planung:

- **Weichensteuerung:** In der nächsten Entwicklungsphase sollen Servomotoren integriert werden, um auch die Weichen direkt über die Weboberfläche schalten zu können.
- **Intelligente Lichtsteuerung:** Die Anlagenbeleuchtung soll in Abhängigkeit von der tatsächlichen Raumbelichtung automatisch oder manuell ein- und ausgeschaltet werden.
- **Architektur-Upgrade:** Um das System noch professioneller aufzustellen, ist geplant, die Webseite auf einen dedizierten Server umzuziehen und die jetzige JSON-Datei durch eine vollwertige SQL-Datenbank abzulösen.

Link zum GitHub-Repository:
[Link zum Quellcode](#)



Zum Scannen mit dem Smartphone